

Understanding the PSPO PRIMALDUALSTEP

A Step-by-Step Explanation with a Concrete Example

1 What is being optimized?

The PRIMALDUALSTEP does not update the reinforcement-learning policy. The RL optimizer has already proposed a new policy before this procedure begins. Instead, the procedure changes the boundaries of a local parameter box

$$\Theta = [\ell, u] = \{\theta : \ell_j \leq \theta_j \leq u_j \text{ for every } j\} \quad (1)$$

around the last certified policy θ_k . This box is the candidate Local Invariant Domain (LID).

There are two competing goals:

1. make the box large in the direction of the proposed policy update; and
2. keep every policy in the box safe on every certified state.

The first goal tries to expand the box, while the second tries to prevent unsafe expansion. The primal–dual procedure balances these goals.

2 The primal problem

Let $\Delta_k = \bar{\theta}_{k+1} - \theta_k$ be the proposed policy update. The primal variables are the box bounds

$$x = (\ell, u). \quad (2)$$

Directional masks freeze the side of every interval that points away from the proposal. For example, if $\Delta_{k,j} > 0$, then

$$\ell_j = \theta_{k,j}, \quad \theta_{k,j} \leq u_j \leq \theta_{k,j} + 10. \quad (3)$$

With weights $w_j = |\Delta_{k,j}|$, the default growth objective is

$$J(\ell, u) = \frac{\sum_j w_j (u_j - \ell_j)}{\sum_j w_j}. \quad (4)$$

PSPO wants to maximize J . Because the constrained optimizer minimizes a loss, it uses

$$f(\ell, u) = -J(\ell, u). \quad (5)$$

Minimizing $-J$ therefore expands the box.

3 The smooth safety constraint

Interval bound propagation (IBP) computes, for each state and action,

$$z_{\theta,a}(s) \in [z_a(s; \Theta), \bar{z}_a(s; \Theta)] \quad \text{for every } \theta \in \Theta. \quad (6)$$

The default all-safe LogSumExp margin is

$$\begin{aligned} \underline{g}_s(\Theta; \tau) = & -\tau \log \sum_{a \in \mathcal{A}_{\text{safe}}(s)} \exp\left(-\frac{z_a(s; \Theta)}{\tau}\right) \\ & - \tau \log \sum_{b \notin \mathcal{A}_{\text{safe}}(s)} \exp\left(\frac{\bar{z}_b(s; \Theta)}{\tau}\right). \end{aligned} \quad (7)$$

The worst-state smooth margin and its constraint violation are

$$\tilde{m}(\Theta) = \min_{s \in \mathcal{S}_{\text{cert}}} \underline{g}_s(\Theta; \tau), \quad \tilde{v}(\Theta) = -\tilde{m}(\Theta). \quad (8)$$

Consequently, $\tilde{v} \leq 0$ means that the smooth constraint is satisfied, while $\tilde{v} > 0$ means that at least one state violates it. The optimization problem is conceptually

$$\min_{\ell, u} -J(\ell, u) \quad \text{subject to} \quad \tilde{v}([\ell, u]) \leq 0. \quad (9)$$

4 Why are there primal and dual variables?

The box bounds (ℓ, u) are the *primal* variables. A nonnegative multiplier λ is the *dual* variable. Intuitively, λ measures how strongly the optimizer should care about safety relative to box growth. A penalty coefficient $\rho > 0$ supplies an additional quadratic cost for violating the smooth constraint.

For an inequality constraint, define

$$\hat{\lambda} = \text{ReLU}(\lambda + \rho \tilde{v}). \quad (10)$$

The augmented-Lagrangian loss minimized with respect to the box bounds is

$$\mathcal{L}_{\text{AL}}(\ell, u; \lambda, \rho) = -J(\ell, u) + \hat{\lambda} \tilde{v} - \frac{(\hat{\lambda} - \lambda)^2}{2\rho}. \quad (11)$$

The gradient of the first term expands the box. When the smooth safety constraint is violated, the remaining terms oppose unsafe expansion.

5 One implementation step

The production implementation uses a differentiable smooth violation for the primal update and a hard, non-differentiable signal for the dual update. Define

$$a^{\text{hard}}(\Theta) = \min_{s \in \mathcal{S}_{\text{cert}}} \mathbf{1} \left[\min_{a \in \mathcal{A}_{\text{safe}}(s)} z_a(s; \Theta) > \max_{b \notin \mathcal{A}_{\text{safe}}(s)} \bar{z}_b(s; \Theta) \right] \in \{0, 1\}, \quad (12)$$

and $v^{\text{hard}} = -a^{\text{hard}}$. One implementation iteration is shown in Algorithm 1.

Algorithm 1 One PSPO primal–dual LID-growth iteration

Require: centre θ_k ; current bounds (ℓ_t, u_t) ; masks (M_ℓ, M_u) ; multiplier λ_t ; penalty ρ_t ; learning-rate scale q_t

- 1: Project (ℓ_t, u_t) onto valid directional bounds around θ_k
- 2: $(\underline{z}, \bar{z}) \leftarrow \text{IBP}([\ell_t, u_t], \mathcal{D}_{\text{sh}})$
- 3: Evaluate J_t , $\tilde{v}_t$, and v_t^{hard}
- 4: Update ρ_t using v_t^{hard}
- 5: $\eta_{p,t} \leftarrow (0.1/n_\theta)q_t$; $\eta_{d,t} \leftarrow 0.1/q_t$
- 6: $\lambda_{t+1} \leftarrow [\lambda_t + \eta_{d,t}\rho_t v_t^{\text{hard}}]_+$ ▷ dual update first
- 7: $\hat{\lambda}_t \leftarrow [\lambda_{t+1} + \rho_t \tilde{v}_t]_+$
- 8: $\mathcal{L}_t \leftarrow -J_t + \hat{\lambda}_t \tilde{v}_t - (\hat{\lambda}_t - \lambda_{t+1})^2 / (2\rho_t)$
- 9: $(\tilde{\ell}_{t+1}, \tilde{u}_{t+1}) \leftarrow \text{ADAMSTEP}((\ell_t, u_t), \nabla \mathcal{L}_t, \eta_{p,t})$ ▷ primal update second
- 10: Update q_t from v_t^{hard}
- 11: $(\ell_{t+1}, u_{t+1}) \leftarrow \text{PROJECTDIRECTIONALBOUNDS}(\tilde{\ell}_{t+1}, \tilde{u}_{t+1}, \theta_k, M_\ell, M_u)$
- 12: **return** $(\ell_{t+1}, u_{t+1}, \lambda_{t+1}, \rho_{t+1}, q_{t+1})$

The update is called *dual–primal* because it updates λ first and then updates (ℓ, u) using the new value of λ . The final projection does not impose the neural safety certificate. It only restores valid box geometry, maximum widths, and directional masks.

6 An important detail of the current implementation

The hard score in Eq. (12) is binary, so

$$v_t^{\text{hard}} = -a_t^{\text{hard}} \in \{-1, 0\}. \quad (13)$$

The multiplier is initialized at $\lambda_0 = 0$ and projected to be nonnegative. It therefore remains zero: a certified box proposes a negative dual step that is clipped to zero, while an uncertified box proposes a zero dual step. The penalty starts at $\rho_0 = 1$ and is increased only if $v_t^{\text{hard}} > 0.05$, which cannot occur for this binary signal. Hence, for the current default implementation,

$$\lambda_t = 0, \quad \rho_t = 1. \quad (14)$$

Substituting these values into Eq. (11) gives the effective primal loss

$$\boxed{\mathcal{L}_{\text{effective}}(\ell, u) = -J(\ell, u) + \frac{1}{2}[\tilde{v}(\ell, u)]_+^2.} \quad (15)$$

Thus the implementation is expressed through a primal–dual API, but its current numerical behavior is a width objective plus a quadratic penalty on the smooth LSE violation.

7 Concrete one-parameter example

Consider a single policy parameter. Let the last safe policy be $\theta_k = 0$ and suppose the proposed policy is $\bar{\theta}_{k+1} = 0.8$. Because the proposal moves in the positive direction, the directional LID has the form

$$\Theta(r) = [0, r], \quad r \geq 0. \quad (16)$$

Here r is the only primal variable and the growth objective is simply

$$J(r) = r. \quad (17)$$

Assume that IBP produces the illustrative worst-state smooth margin

$$\tilde{m}(r) = 0.6 - r. \quad (18)$$

The smooth violation is therefore

$$\tilde{v}(r) = r - 0.6. \quad (19)$$

The box is hard-certified only when $r < 0.6$. Using Eq. (15), the loss is

$$\mathcal{L}(r) = -r + \frac{1}{2}[r - 0.6]_+^2, \quad (20)$$

with derivative

$$\frac{d\mathcal{L}}{dr} = \begin{cases} -1, & r \leq 0.6, \\ -1 + (r - 0.6), & r > 0.6. \end{cases} \quad (21)$$

For transparent arithmetic, replace Adam by ordinary gradient descent with step size $\eta = 0.25$. This simplification changes the numerical trajectory, but not the competition between growth and safety. The update is

$$r_{t+1} = \text{clip}_{[0,10]} \left(r_t - 0.25 \frac{d\mathcal{L}}{dr}(r_t) \right). \quad (22)$$

t	r_t	$\tilde{v}_t$	$d\mathcal{L}/dr$	r_{t+1}	Hard-certified?
0	0.000	-0.600	-1.000	0.250	yes
1	0.250	-0.350	-1.000	0.500	yes
2	0.500	-0.100	-1.000	0.750	yes
3	0.750	0.150	-0.850	0.963	no
4	0.963	0.363	-0.638	1.122	no
5	1.122	0.522	-0.478	1.241	no

The first term $-r$ continually tries to increase the interval. Once $r > 0.6$, the quadratic penalty pushes in the opposite direction, but initially it is not strong enough to keep the optimizer inside the safe region. In this example the continuous optimum of the penalized objective is $r = 1.6$, which is not hard-certified. This is not a safety failure because the optimization trajectory is only a search for useful candidate boxes.

Suppose a certificate is evaluated after every illustrative step. The last certified checkpoint is

$$\Theta^* = [0, 0.5]. \quad (23)$$

The proposed policy parameter 0.8 is then projected into this box:

$$\theta_{k+1} = \text{Proj}_{[0,0.5]}(0.8) = \min\{0.5, \max\{0, 0.8\}\} = 0.5. \quad (24)$$

Although later optimization iterates produced larger unsafe boxes, they are discarded. Only the hard-certified checkpoint is used for the policy update.

8 Where does the safety guarantee come from?

The primal-dual step is a heuristic for finding a large box. It is not the proof of safety. PSPO obtains the guarantee through a separate checkpoint procedure:

1. save candidate boxes during LID optimization;
2. rebuild IBP bounds for each saved box on the exhaustive certificate dataset;
3. retain only boxes satisfying the strict hard logit ordering at every state;
4. project the proposed policy into the last retained box; and
5. revert to θ_k if no box qualifies.

The optimizer may therefore visit infeasible boxes without endangering the returned policy. The hard checkpoint filter, projection, and fail-closed reversion are the safety-critical operations.

9 Summary

One PRIMALDUALSTEP can be understood as follows:

$$\begin{array}{l} \text{measure box width and worst-state safety} \longrightarrow \\ \quad \text{adjust the safety multiplier} \longrightarrow \\ \quad \text{take one Adam step on the box bounds} \longrightarrow \\ \text{restore directional and geometric constraints.} \end{array} \tag{25}$$

Repeating these steps generates candidate LIDs. A separate exhaustive hard certificate decides which, if any, may be used to update the policy.